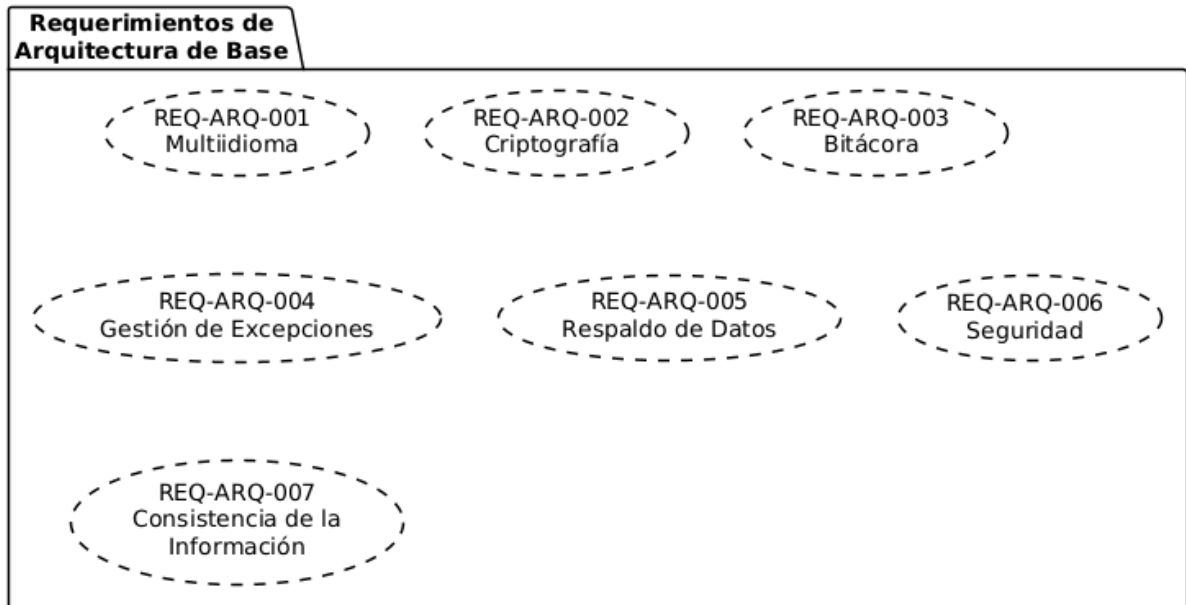


1.3 Requerimientos de Arquitectura de Base

1.3 - Requerimientos de Arquitectura de Base



Propósito

Estos 7 requerimientos definen los servicios transversales de la arquitectura de base que deben ser implementados como un proyecto .NET independiente y reutilizable. A diferencia de los requerimientos funcionales y no funcionales, estos especifican mecanismos técnicos obligatorios que el sistema debe incorporar para garantizar seguridad, trazabilidad, mantenibilidad y calidad del código. Su implementación como biblioteca de servicios permite su reutilización en futuros desarrollos del centro de salud.

REQ-ARQ-001: Multiidioma (i18n)

Introducción: El sistema debe proporcionar un mecanismo de internacionalización que permita adaptar la interfaz de usuario, mensajes y documentación al idioma del usuario sin necesidad de recompilar el código fuente.

Entradas: Archivos de recursos (.resx) con pares clave-valor para cada idioma soportado (es-AR, en-US, etc.); cultura preferida del usuario (desde su perfil o configuración del sistema operativo).

Procesos: Implementar una clase `LocalizationService` que exponga un método `ObtenerTexto(string clave)`; al iniciar la aplicación, determinar la cultura activa (perfil del usuario o defecto es-AR); cargar el `ResourceManager` correspondiente y cachearlo; si una clave no existe en el idioma solicitado, devolver el valor por defecto (es-AR); permitir cambio de idioma en caliente mediante recarga del `ResourceManager`.

Salidas: Interfaz de usuario completamente traducida al idioma configurado; posibilidad de incorporar un nuevo idioma agregando únicamente un nuevo archivo .resx sin modificar la lógica de negocio.

REQ-ARQ-002: Criptografía

Introducción: El sistema debe ofrecer un módulo de cifrado reutilizable que provea funciones de encriptación/desencryptación de datos, hashing seguro de contraseñas y generación de claves, utilizando algoritmos estándar (AES-256, SHA-256/bcrypt).

Entradas: Datos en texto plano a cifrar (valores de RIN, datos personales, contraseñas); configuración de algoritmos (tipo, longitud de clave, vector de inicialización).

Procesos: Implementar una clase `CryptographyService` con métodos estáticos: `Encriptar(string textoPlano)` (retorna texto cifrado en Base64 usando AES-256), `Desencriptar(string textoCifrado)` (revierte el proceso), `Hashear(string texto)` (genera hash unidireccional SHA-256 o bcrypt), `VerificarHash(string texto, string hash)` (compara texto plano con hash almacenado); la clave maestra debe almacenarse de forma segura (fuera del código fuente); utilizar `System.Security.Cryptography` de .NET Framework.

Salidas: Biblioteca de clases `Services` con el módulo de criptografía compilado; datos sensibles almacenados en la base de datos únicamente en formato cifrado o hasheado; verificación de integridad de contraseñas sin almacenarlas en texto plano.

REQ-ARQ-003: Bitácora (Logging)

Introducción: El sistema debe incorporar un módulo de bitácora que permita a todas las capas registrar eventos relevantes: inicios de sesión, operaciones de escritura, errores, advertencias y mensajes informativos, proporcionando trazabilidad uniforme y facilitando el diagnóstico de fallos.

Entradas: Solicitudes de registro desde cualquier punto del código con: nivel de severidad (Debug, Info, Warning, Error, Fatal), mensaje descriptivo, opcionalmente excepción capturada, identificador del usuario, nombre de la capa de origen, fecha y hora.

Procesos: Implementar una clase `BitacoraService` con método estático `Registrar(Nivel nivel, string mensaje, Exception ex = null, string usuario = "")`; formatear el mensaje incluyendo timestamp, nivel, mensaje, usuario y stack trace (si hay excepción); delegar la persistencia a la capa DAL insertando en una tabla `Logs`; configurar el nivel mínimo de registro vía `app.config`; si la escritura en BD falla, escribir en archivo de fallback (`log_fallback.txt`) con rotación automática a 10 MB; asegurar que el método no lance excepciones hacia el invocante.

Salidas: Registros de eventos almacenados en la tabla `Logs` (o archivo de fallback); trazabilidad completa disponible para consulta del administrador; capacidad de diagnosticar fallos y auditar acciones de usuarios.

REQ-ARQ-004: Gestión de Excepciones

Introducción: El sistema debe proporcionar un módulo centralizado de gestión de excepciones que capture, registre y notifique cualquier error no controlado en las capas de presentación (WinForms), lógica de negocio (BLL) o acceso a datos (DAL), garantizando que ningún error deje de ser registrado y que el usuario final reciba siempre un mensaje amigable.

Entradas: Excepción no controlada (objeto `Exception`) capturada por el manejador global o por bloques `try-catch`; contexto opcional (nombre del método o capa); política configurable de reintentos para errores transitorios.

Procesos: Implementar una clase `ExceptionHandler` con métodos estáticos: `ManejarExcepcion(Exception ex, string contexto = "", bool mostrarMensaje = true)` (registra en bitácora y muestra diálogo amigable al usuario), `Reintentar<T>(Func<T> operacion, int maxReintentos = 3, int intervaloMs = 1000)` (ejecuta operación con reintentos ante fallos transitorios); suscribir el método a los eventos globales de WinForms (`Application.ThreadException`) y del dominio (`AppDomain.CurrentDomain.UnhandledException`); envolver `SQLException` en excepción personalizada `DataAccessException` que oculte detalles de conexión; asegurar que el `ExceptionHandler` nunca lance una excepción secundaria.

Salidas: Todas las excepciones no controladas registradas automáticamente en la bitácora; usuario final expuesto únicamente a mensajes de error amigables; mecanismo de reintentos disponible para operaciones transitorias.

REQ-ARQ-005: Respaldo de Datos

Introducción: El sistema debe proporcionar un módulo reutilizable de respaldo de datos que permita crear, restaurar y verificar copias de seguridad de la base de datos SQL Server, encapsulando la lógica de los comandos SQL nativos y exponiéndolos a través de la fachada de la arquitectura de base.

Entradas: Ruta de destino para el archivo de respaldo (.bak); tipo de respaldo (Completo/FULL o Diferencial/DIFFERENTIAL); credenciales de conexión a la base de datos; opcionalmente nombre de la base de datos.

Procesos: Implementar una clase `BackupService` con métodos:

`RealizarBackup(TipoBackup tipo)` (ejecuta `BACKUP DATABASE` y genera el archivo .bak, registrando en bitácora), `RestaurarBackup(string rutaArchivo)` (ejecuta `RESTORE DATABASE` previa verificación de que la BD no esté en uso), `VerificarIntegridadBackup(string rutaArchivo)` (ejecuta `RESTORE VERIFYONLY`); la capa DAL ejecutará los comandos SQL nativos con permisos de administrador; antes de cada respaldo, verificar espacio suficiente en disco; programar tarea automática diaria (ej. 3:00 AM) que invoque `RealizarBackup(TipoBackup.Completo)`.

Salidas: Archivo .bak generado en la ruta configurada con nombre que incluya fecha y tipo (ej. `HematoDB_20260430_FULL.bak`); registro en la tabla Logs de cada operación; confirmación a la capa invocante del resultado.

REQ-ARQ-006: Seguridad (Autenticación y Autorización)

Introducción: El sistema debe contar con un módulo de seguridad que centralice la lógica de autenticación y autorización de usuarios, consumido por el formulario de login y por cualquier otro sistema que requiera validar credenciales y verificar permisos.

Entradas: Nombre de usuario y contraseña en texto plano (ingresados en el login); solicitud de verificación de permiso (identificador del usuario y nombre del permiso/rol requerido).

Procesos: Implementar una clase `SeguridadService` con métodos estáticos: `Autenticar(string usuario, string contraseña)` (busca al usuario en BD, verifica el hash de la contraseña usando `CryptographyService`, registra inicio de sesión en bitácora y retorna objeto `UsuarioAutenticado`), `TienePermiso(int idUsuario, string permiso)` (consulta roles y permisos del usuario, retorna true si posee el permiso); definir la clase `UsuarioAutenticado` en `DomainModel` con `Id`, `NombreUsuario`, `NombreCompleto`, `Perfil`; prevenir ataques de fuerza bruta: tras 5 intentos fallidos consecutivos, bloquear la cuenta temporalmente por 15 minutos; las contraseñas nunca deben transmitirse ni almacenarse en texto plano.

Salidas: Sesión iniciada con objeto `UsuarioAutenticado` disponible para toda la aplicación; registro de auditoría de todos los inicios de sesión (exitosos y fallidos); mecanismo de bloqueo temporal ante fuerza bruta.

REQ-ARQ-007: Consistencia de la Información

Introducción: El sistema debe garantizar la integridad y consistencia de la información que persiste la aplicación, evitando corrupción de datos, duplicidades no permitidas, violaciones de integridad referencial y condiciones de carrera en operaciones concurrentes.

Entradas: Solicitud de validación desde la BLL antes de ejecutar una operación de inserción, actualización o borrado; entidad o conjunto de entidades a validar; reglas de consistencia a aplicar (unicidad de documento, existencia de claves foráneas, rangos numéricos válidos, fechas coherentes).

Procesos: Implementar una clase `ConsistenciaService` con métodos:

`ValidarEntidad<T>(T entidad)` (utiliza reflexión y atributos personalizados [Requerido], [Unico], [Rango(min, max)] para validar, retorna true/false y llena lista de mensajes de error), `IniciarTransaccion()`, `ConfirmarTransaccion()`, `RevertirTransaccion()`; definir atributos personalizados en `DomainModel`; todos los métodos de escritura en la DAL deben ejecutarse dentro de una transacción iniciada por la BLL; en caso de error de validación o excepción, llamar automáticamente a `RevertirTransaccion()`; registrar toda validación fallida en bitácora con nivel Warning.

Salidas: Datos almacenados en la base de datos siempre consistentes y libres de corrupción; transacciones atómicas (ante cualquier error, la BD retorna a su estado previo); lista de mensajes de error de validación disponible para mostrar al usuario.